23BCS11290
Sambhav Mahajan

Q Flights[i] = [from$_i$, to$_i$, price$_i$]
Cheapest flight from src, dst, and k, return the cheapest price from src to dst with at most k steps

Algorithm / Approach
(i) Create an adjacency matrix for given graph.
(ii) Create queue and push (src, 0)
(iii) ~~Create queue~~ Set t = 0, dist[n] = INF
(iv) While queue not empty and t ≤ k
→ Set sz = q.size();
→ While sz > 0
node, cost = q.pop();
→ for each child of node
if the dist[child] is greater than dist[node] + cost :
update dist[child]
sz - -;
t++;

if dist[dst] != INF :
return dist[dst]
return -1

Page No.:
Date:

pseudocode :

```
int helper (flights, src, dst, k, n) {

    vector<vector<pair<int,int>> g(n);
    for (auto f: flights) {
        g[f[0]].push_back({f[1], f[2]})
    }

    vector<int> dist(n, 1e8);
    queue<pair<int, int>> q;
    q.push({src, 0});

    int t = 0;
    while (!q.empty() && t <= k) {
        int sz = q.size();
        while (sz-- > 0) {
            auto f = q.front(); q.pop();
            for (auto p : g[f.first])
            {
                if (dist[p.first] >
                    dist[f.first] + p.second) {
                    dist[p.first] = dist[f.first] +
                        p.second;
                }
            }
        }
        t++;
    }
```

```
if (dist[dst] != 1e8) return dist[dst];
return -1;
```

Time Complexity: $O(V + EK)$
Space Complexity: $O(V + E)$

I/O

I: [[0, 1, 100], [1, 2, 100], [2, 0, 100], [1, 3, 600], [2, 3, 200]], src = 0
K = 1

Output: 700